

OpenHospital API Network Security Lab

Giuseppe Pisano
g.pisano@unibo.it

Healthcare Data Privacy and Security

August, 2026



S.K.I.L.L.E.D — AID013244/04/2

Strategic Knowledge and Inclusive Lifelong Learning in the health sector for youth Employability and Development

- 1 Lab Goal
- 2 Deploy OpenHospital API
- 3 Open Proxy Baseline
- 4 CRUD API Exercise
- 5 Subnet and Path Policy
- 6 Verification
- 7 Logs and Investigation

8 Final Integration

Learning path

We deploy the real **OpenHospital API**, put **Nginx** in front of it with a deliberately open policy, prove that login and API endpoints are reachable, and then harden the proxy using **subnets** and **API paths**.

Hospital storyline

St. Isidore Hospital exposes OpenHospital API for internal clinical systems. Clinical workstations, the lab, remote VPN users, and guest Wi-Fi should not all have the same network reachability.

Where this fits

Lesson 04 studied OpenHospital users, groups, permissions, and logs. Lesson 06 studied networks, firewalls, proxies, and VPNs. This lab connects them with a real healthcare API.

- **Authentication:** API login returns a token.
- **Access control:** OpenHospital still decides what the logged-in user can do.
- **Network control:** Nginx decides which subnet can reach which API area.
- **Evidence:** proxy logs and application logs answer different questions.

Lab Goal

Section Context: Lab Goal

What we are talking about

OpenHospital API exposes REST endpoints for OpenHospital business logic implemented in OpenHospital Core [OpenHospital API, 2026]. The lab uses that real API, not a mock service.

Hospital lens

The question becomes realistic: once an API exists, which hospital subnetworks should reach login, patient endpoints, laboratory endpoints, Swagger documentation, and health checks?

What Must Be Done

Lab tasks

- prepare OpenHospital API source and configuration;
- build the database and backend containers;
- start Nginx with an open proxy policy;
- test healthcheck, DMZ Swagger access, login, CRUD endpoints, and read-only clinical data;
- replace the open policy with subnet/path restrictions;
- verify allowed and denied flows from different client containers.

Why Start with an Open Proxy

Engineering reason

Security hardening is easier when the baseline deployment is known to work. First prove that the API, database, proxy, and clients can communicate. Then change one security rule at a time.

Hospital example

If the lab subnet cannot reach `/laboratories`, first determine whether OpenHospital API is down, Nginx cannot route, authentication failed, or the subnet rule blocks the request.

Final Policy Target

Network	Allowed API Area	Reason
Clinical	patients, admissions, admission types, lab APIs	clinical care needs broad access
Lab	laboratories, exams, exam types, exam rows	laboratory workflow, not full patient browsing
VPN	clinical APIs	remote clinical work through controlled access
DMZ	Swagger, OpenAPI docs	documentation and API testing are isolated
Guest	none	patient Wi-Fi is isolated from hospital systems

Recap: Lab Goal

Takeaways

- The lab uses the real OpenHospital API project.
- The first working state is intentionally permissive.
- The second state adds network restrictions that match hospital roles.

In St. Isidore Hospital

St. Isidore first deploys the system, then turns reachability into explicit policy.

Deploy OpenHospital API

Section Context: Deployment

What we are talking about

The official API build requires OpenHospital Core, API configuration files, a database, and a Java backend [OpenHospital API, 2026]. The lab script prepares those pieces under `vendor/`.

Hospital lens

This mirrors a realistic operational sequence: configure dependencies, start the database, initialize data, start the API, and only then publish it through a proxy.

Prepare the Source

Command

```
cd lesson-07/examples/lab  
./prepare-openhospital-api.sh
```

What happens

The script clones `informatici/openhospital-api`, copies the lab `.env`, and runs the upstream `make` target that prepares OpenHospital Core and configuration files.

Build and Start Database

Commands

```
docker compose build database backend  
docker compose up -d database  
docker compose run --rm oh-database-init
```

Expected observation

The first build can take several minutes because Maven downloads dependencies and compiles OpenHospital Core and API.

Start API, Proxy, and Clients

Command

```
docker compose up -d backend reverse-proxy \  
  clinical-client lab-client guest-client vpn-client \  
  dmz-client
```

Expected observation

backend runs OpenHospital API. reverse-proxy publishes port 8088 on the host and forwards to the backend.

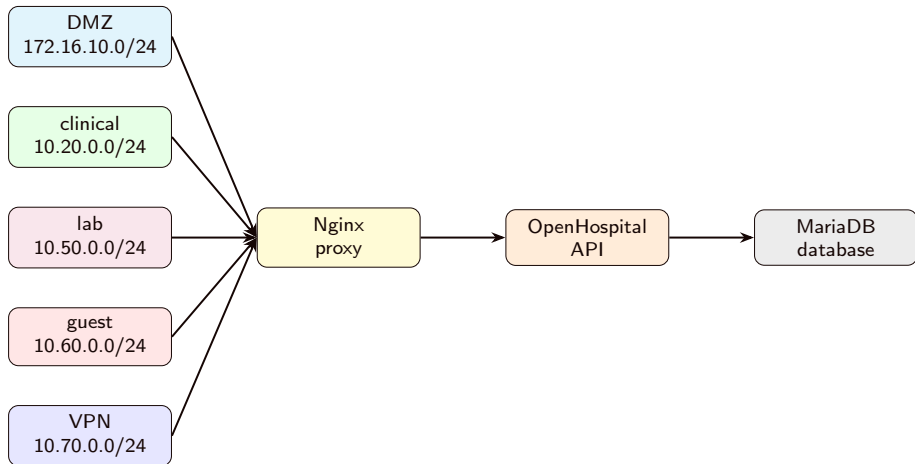
Inspect Running Services

Commands

```
docker compose ps
```

```
docker compose logs --tail=100 backend
```

Deployment Scheme



Recap: Deploy OpenHospital API

Takeaways

- Real deployment adds build, database, and startup dependencies.
- Logs are essential during startup.
- The proxy should be tested only after the backend is healthy.

In St. Isidore Hospital

St. Isidore treats deployment as part of security: an unavailable or misconfigured API cannot be secured by proxy rules alone.

Open Proxy Baseline

Section Context: Open Proxy Baseline

What we are talking about

The starter Nginx policy forwards every request to OpenHospital API. This is intentionally insecure, but useful as a baseline.

Hospital lens

At this stage guest Wi-Fi can still reach the API. That is not the final policy; it is the problem we will fix after proving the deployment works.

Starter Proxy Policy

Nginx

```
location / {  
    proxy_pass http://backend:8080;  
}
```

Test Healthcheck

Commands

```
docker compose exec clinical-client \  
    curl -i http://oh-proxy/healthcheck  
docker compose exec guest-client \  
    curl -i http://oh-proxy/healthcheck
```

Reading the result

Both should reach the backend in the open baseline. Guest success is evidence that hardening is still missing.

Test Swagger from the DMZ

Command

```
docker compose exec dmz-client \  
  curl -i http://oh-proxy/swagger-ui/index.html
```

Why Swagger matters

Swagger confirms that the API is alive and shows endpoint categories such as patients, laboratories, admissions, users, and authentication.

Why Swagger Belongs in a DMZ-Like Network

Principle

Swagger and OpenAPI metadata describe the shape of the system. They are useful for operators, developers, and testers, but they also help an attacker map available operations.

Hospital example

St. Isidore lets a DMZ documentation workstation inspect API contracts. Clinical workstations use the API they need, but they do not need interactive API documentation during routine care.

Test Login

Command

```
docker compose exec clinical-client curl -i \  
  -H "Content-Type: application/json" \  
  -d '{"username":"admin","password":"admin"}' \  
  http://oh-proxy/auth/login
```

Login and Network Control

Important distinction

A successful login proves that application authentication works. It does not prove that the network exposure is acceptable.

Hospital example

If a guest Wi-Fi device can reach `/auth/login`, the hospital is letting unmanaged devices attempt application authentication. The proxy should block that path earlier.

Recap: Open Proxy Baseline

Takeaways

- The open proxy baseline proves that the system works.
- It also proves that guest reachability exists and must be removed.
- Authentication and network exposure are separate layers.

In St. Isidore Hospital

St. Isidore uses the open baseline only long enough to confirm that the API, database, and proxy are functioning.

CRUD API Exercise

Section Context: CRUD API Exercise

What we are talking about

Status-code checks prove reachability. CRUD tests prove that the API can perform normal work: **create**, **read**, **update**, and **delete** a resource.

Hospital lens

The lab uses `/admissiontypes`, a small administrative resource, so the exercise changes disposable training data instead of patient records.

Run the CRUD Script

Command

```
./crud-admissiontypes.sh
```

What the script does

It logs in as admin, extracts the JWT token, reads admission types, creates code ZZ, updates its description, reads again, and deletes it.

Call Policy APIs Easily

One endpoint

```
./api-call.sh clinical GET /patients?page=0\&size=3  
./api-call.sh lab GET /examtypes  
OH_NO_AUTH=1 ./api-call.sh dmz GET /swagger-ui/index.html
```

Guided tour

`./crud-policy-apis.sh` prints responses for the API paths used in the lab policy, including allowed calls, denied subnet calls, successful CRUD, and verified reads against real demo data.

What Is Actually CRUD?

Verified write workflows

The guided lab performs real create/read/update/delete work only where the OpenHospital API accepts the complete object model without extra clinical workflow setup.

Hospital lens

St. Isidore changes low-risk training records: admission-type configuration, lab exam-type configuration, and a temporary patient created only for the exercise.

Administrative CRUD Requests

API operations

```
GET      /admissiontypes
POST     /admissiontypes
PUT      /admissiontypes
DELETE   /admissiontypes/ZZ
```

Network-security meaning

These are not just web pages. They are operations that can change hospital configuration data, so the proxy policy must decide which subnets can reach them.

Lab Configuration CRUD Requests

API operations

```
GET /examtypes
POST /examtypes
PUT /examtypes/ZT
DELETE /examtypes/ZT
GET /examtypes
```

Network-security meaning

The lab subnet can maintain laboratory exam-type configuration, but it still cannot access general clinical administration endpoints such as /admissiontypes.

Patient CRUD Requests

API operations

```
POST /patients
GET /patients/{created-code}
PUT /patients/{created-code}
GET /patients/{created-code}
DELETE /patients/{created-code}
```

Hospital lens

The script creates a temporary training patient named Security Patient, updates it, checks the result, and deletes it.

Verified Read-Only API Areas

Real demo-data reads

```
GET /exams/description/WBC
GET /examrows/byExamCode/01.02
GET /admissions?...page=0&size=3
GET /laboratories?oneWeek=false&page=0&size=3
GET /laboratories/15
```

Why not fake writes here?

OpenHospital exams and laboratory requests are tied to existing clinical workflows and database constraints. The lab therefore reads real records instead of inventing invalid objects.

Example Payloads

Create and update

```
{"code": "ZZ", "description": "SECURITY LAB TRIAGE"}
```

```
{"code": "ZZ", "description": "SECURITY LAB TRIAGE UPDATED"}
```

Recap: CRUD API Exercise

Takeaways

- CRUD tests confirm useful API behavior where the model is simple enough for the lab.
- Complex clinical objects are tested through verified reads against real demo data.
- The JWT proves application authentication is active.
- The same endpoint can later be allowed for clinical/VPN and denied for lab/guest.

In St. Isidore Hospital

St. Isidore tests a harmless administrative record before applying network restrictions to API operations.

Subnet and Path Policy

Section Context: Subnet and Path Policy

What we are talking about

The hardened policy combines **where the request comes from** with **which API path it targets**. Nginx access rules can enforce this before OpenHospital API processes the request [NGINX, 2026].

Install the Hardened Policy

Commands

```
cp nginx/solution.conf nginx/default.conf  
docker compose exec reverse-proxy nginx -s reload
```

Guest Network Rule

Policy idea

```
deny all;
```

Hospital meaning

Guest Wi-Fi is for patients and visitors. It should not reach OpenHospital API, Swagger, patient records, admissions, or laboratory workflows.

Laboratory API Rule

Policy idea

```
location ~/laboratories {  
    allow 10.50.0.0/24;  
    deny all;  
    proxy_pass http://backend:8080;  
}
```

Lab Network Is Not Full Clinical Access

Principle

The laboratory subnet needs laboratory workflows, not unrestricted access to patient administration or user management endpoints.

Hospital example

The lab can reach laboratory API paths such as `/laboratories`, and the tutorial verifies this with real read requests. It should not browse all `/patients` endpoints just because it is inside the hospital.

Clinical and VPN Rules



Policy idea

```
location ~~ /patients {  
    allow 10.20.0.0/24;  
    allow 10.70.0.0/24;  
    deny all;  
    proxy_pass http://backend:8080;  
}
```

VPN Is a Controlled Clinical Path

Principle

VPN access should not mean access to every internal system. In this lab, VPN users can reach clinical API paths through the proxy, but policy can still restrict documentation or administration.

CRUD Administrative API Rule

Policy idea

```
location ~/admissiontypes {  
    allow 10.20.0.0/24;  
    allow 10.70.0.0/24;  
    deny all;  
    proxy_pass http://backend:8080;  
}
```


CRUD Access Is Still Least Privilege

Principle

An authenticated user is not enough. The request should also arrive from a network where that operation makes sense.

Hospital example

Clinical administration can maintain admission-type configuration. The laboratory subnet should not change admission workflow metadata just because lab instruments need lab APIs.

DMZ Swagger Rule

Policy idea

```
location ~/swagger-ui {  
    allow 172.16.10.0/24;  
    deny all;  
    proxy_pass http://backend:8080;  
}
```

Hospital meaning

The DMZ-like documentation network can inspect API contracts. Clinical, lab, guest, and VPN clients test concrete endpoints instead of browsing the full API map.

Recap: Subnet and Path Policy

Takeaways

- Subnet rules answer “from where?”
- Path rules answer “to which function?”
- Least privilege applies to networks, not only users.

In St. Isidore Hospital

St. Isidore gives the lab subnet laboratory reachability without turning it into a general clinical network.

Verification

Section Context: Verification

What we are talking about

The hardened policy is only credible if each subnet is tested. A blocked request can be a successful security result.

Manual Tests

Commands

```
docker compose exec lab-client \  
    curl -i http://oh-proxy/laboratories  
docker compose exec lab-client \  
    curl -i http://oh-proxy/patients  
docker compose exec guest-client \  
    curl -i http://oh-proxy/healthcheck  
docker compose exec dmz-client \  
    curl -i http://oh-proxy/swagger-ui/index.html  
docker compose exec clinical-client \  
    curl -i http://oh-proxy/swagger-ui/index.html
```

Expected Meaning

Request	Expected	Meaning
Lab to /laboratories	allowed	lab workflow path works
Lab to /patients	403	lab is not full clinical network
Guest to /healthcheck	403	guest Wi-Fi is isolated
Clinical to /patients	allowed	clinical path works
VPN to /patients	allowed	remote clinical path works
DMZ to Swagger	allowed	documentation network can inspect API docs
Clinical to Swagger	403	care network does not expose API map
Clinical to CRUD API	allowed	administrative API path works
Lab to CRUD API	403	lab subnet is not administrative

Automated Check



Command

```
./check-policy.sh
```

Reading status codes

Nginx 403 means the proxy blocked the request. 200 or 401 can both mean the request reached OpenHospital API, depending on whether the endpoint requires authentication.

Application Status vs Proxy Status

Important

Network policy testing and application authorization testing are related but not identical.

Hospital example

If the lab subnet receives 403 for /patients, the proxy blocked it. If it receives 401, the request reached OpenHospital API and the application is asking for authentication.

Recap: Verification

Takeaways

- Test from every relevant subnet.
- Interpret status codes by layer.
- A good lab result includes command output and explanation.

In St. Isidore Hospital

St. Isidore does not trust policy by inspection alone; it proves the behavior from each network.

Logs and Investigation

Section Context: Logs

What we are talking about

The proxy and OpenHospital API produce different evidence. Both are needed to understand what happened.

Read Proxy Logs

Command

```
docker compose logs --tail=50 reverse-proxy
```

Read Backend Logs

Command

```
docker compose logs --tail=100 backend
```

What Each Log Answers

Log	Question it helps answer
Proxy log	Which subnet requested which path, and was it allowed or denied?
Backend log	Did the request reach OpenHospital API, and did the API report errors?
Application audit	Which authenticated user performed a clinical action?

Design One New Rule

Exercise

Choose one additional policy: allow lab access to `/patients/search`, deny VPN access to Swagger, allow guest access only to `/healthcheck`, or create an admin subnet for `/users` and `/usergroups`.

Evidence

Record the rule, test command, observed status code, and one proxy log line.

Recap: Logs and Investigation

Takeaways

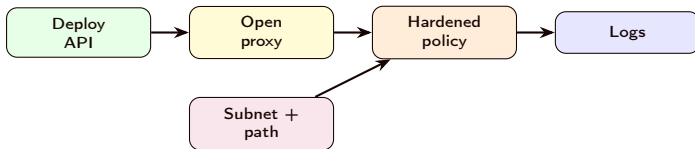
- Proxy logs show perimeter decisions.
- Backend logs show application processing.
- New rules should be justified by workflow, not convenience.

In St. Isidore Hospital

St. Isidore can explain both blocked guest traffic and allowed clinical API use with evidence.

Final Integration

Final Concept Map



Reading the map

Deployment proves the API works. The open proxy proves routing. The hardened policy limits access by source network and API area. Logs provide evidence for the result.

Final Recap



Core lesson

Network security around a healthcare API is not just “put it behind Nginx”. The proxy must encode clinical workflow decisions: guest devices get no access, lab systems get lab APIs, and clinical or VPN networks get broader but still controlled access.

In St. Isidore Hospital

OpenHospital API remains the application authority, but Nginx reduces who can reach each part of the API before credentials or tokens are even considered.

References I

[NGINX, 2026] NGINX (2026).

Module ngx_http_access_module.

NGINX

https://nginx.org/en/docs/http/ngx_http_access_module.html.

[OpenHospital API, 2026] OpenHospital API (2026).

Open Hospital API Documentation.

Informatici Senza Frontiere

<https://oh2.open-hospital.org/oh20-developer-portal/api/installation/>.